

CS1CA Computer Systems Architecture

Processor Pipelining (2)

Week 8 lecture 2 of autumn semester 2025

Lectured by

Dr. Hong Wei

h.wei@reading.ac.uk

Room 133 in Polly Vacher

Instr No.	Pipeline Stage						
1	IF	ID	EX	MEM	WB		
2		IF	ID	EX	MEM	WB	
3			IF	ID	EX	MEM	WB
4				IF	ID	EX	MEM
5					IF	ID	EX
Clock Cycle	1	2	3	4	5	6	7

Outline of the topic

- Introduction to pipelining
- The basic pipeline for DLX machine
- Performance issues in pipelining
- Pipeline hazards
 - Data hazards
 - Structure hazards
 - Control hazards
- DLX pipeline to handle multi-cycle operations
- Summary: the MIPS R4000 pipeline (self-learning)

In this lecture

- The following contents will be delivered
 - Performance issues in pipelining
 - Pipeline hazards
 - Data hazards
- The learning outcomes are listed below
 - To be able to calculate the performance gain from pipelining
 - To be able to identify data hazards from pipeline timing and deal with pipeline data hazards by using introduced techniques

Performance issues in pipelining

- Pipelining increases instruction throughput (reduce average CPI);
- Pipelining does not reduce the execution time of an individual instruction;
- Pipeline stage imbalance reduces performance: the slowest pipeline stage determines pipeline clock cycle time ;
- Pipeline overhead increases the execution time of each instruction.

5-stage pipeline for DLX (RICS / MIPS)

- pipeline timing table

Clk. no. \ Inst. no. i	1	2	3	4	5	6	7	8	9
i	IF	ID	EX	MEM	WB				
$i+1$		IF	ID	EX	MEM	WB			
$i+2$			IF	ID	EX	MEM	WB		
$i+3$				IF	ID	EX	MEM	WB	
$i+4$					IF	ID	EX	MEM	WB

A benchmark runs in a pipelined machine. Ideal CPI is 1.

Example: calculate average CPI

- In an un-pipelined DLX machine, a benchmark program is under running. In the benchmark,
 - Branch instructions takes 12% (frequency) of all instructions, and store instruction's frequency is 5%.
 - CPI for both branch and store instructions is 4 clock cycles, and CPI for other instructions is 5 clock cycles.

$$\begin{aligned}\text{average CPI} &= \sum_{i=1}^3 (\text{CPI}_i \times \text{frequency}_i) \\ &= (4 \times 12\%) + (4 \times 5\%) + (5 \times 83\%) = 4.83\end{aligned}$$

- For a pipelined DLX machine, ideally the average CPI is 1 for executing the benchmark program.
- Assuming there is no other change of the system, the performance gain (**speedup**) from pipelining is **4.83**.

Pipeline hazards

- Pipeline hazards are situations that prevent the next instruction in the instruction stream from executing during its designated clock cycle.
- There are three classes of hazards
 - Structural hazards
 - Data hazards
 - Control hazards
- It is necessary to **stall** a pipeline to avoid hazards occurring in the pipeline.

Performance of pipelines with stalls

$$\text{Speedup from pipelining} = \frac{\text{CPU time}_{\text{unpipelined}}}{\text{CPU time}_{\text{pipelined}}}$$

$$= \frac{IC_{\text{unpipelined}} \times CPI_{\text{unpipelined}} \times \text{clock cycle time}_{\text{unpipelined}}}{IC_{\text{pipelined}} \times CPI_{\text{pipelined}} \times \text{clock cycle time}_{\text{pipelined}}} \quad (1)$$

- Pipelining doesn't affect IC so that $IC_{\text{pipelined}} = IC_{\text{unpipelined}}$

$$CPI_{\text{pipelined}} = \text{Ideal CPI} + \text{Pipeline stall cycles per instruction}$$

- **Hypothesis**: Ignoring the overhead introduced by pipelining, and assuming the stages are perfectly balanced, then the clock cycle time of the two machines can be equal.

Equation (1) can be written as

$$\text{speedup from pipelineing} = \frac{CPI_{unpipelined}}{1 + \text{Pipeline stall cycles per instruction}}$$

If all instructions take the same number of stages (also called pipeline depth) in execution, we have

$$\text{Speedup from pipelining} = \frac{\text{Pipeline depth}}{1 + \text{Pipeline stall cycles per instruction}}$$

Pipeline depth refers to the number of pipeline stages

Pipeline Hazards - revisit

Hazards are situations that prevent the next instruction in the instruction stream from executing during its designated clock cycle.

There are three types of hazards

- Data hazards
- Structural hazards
- Control hazards

Let's start our discussion with Data Hazards, the most popular problem in pipelined machine.

Data Hazards

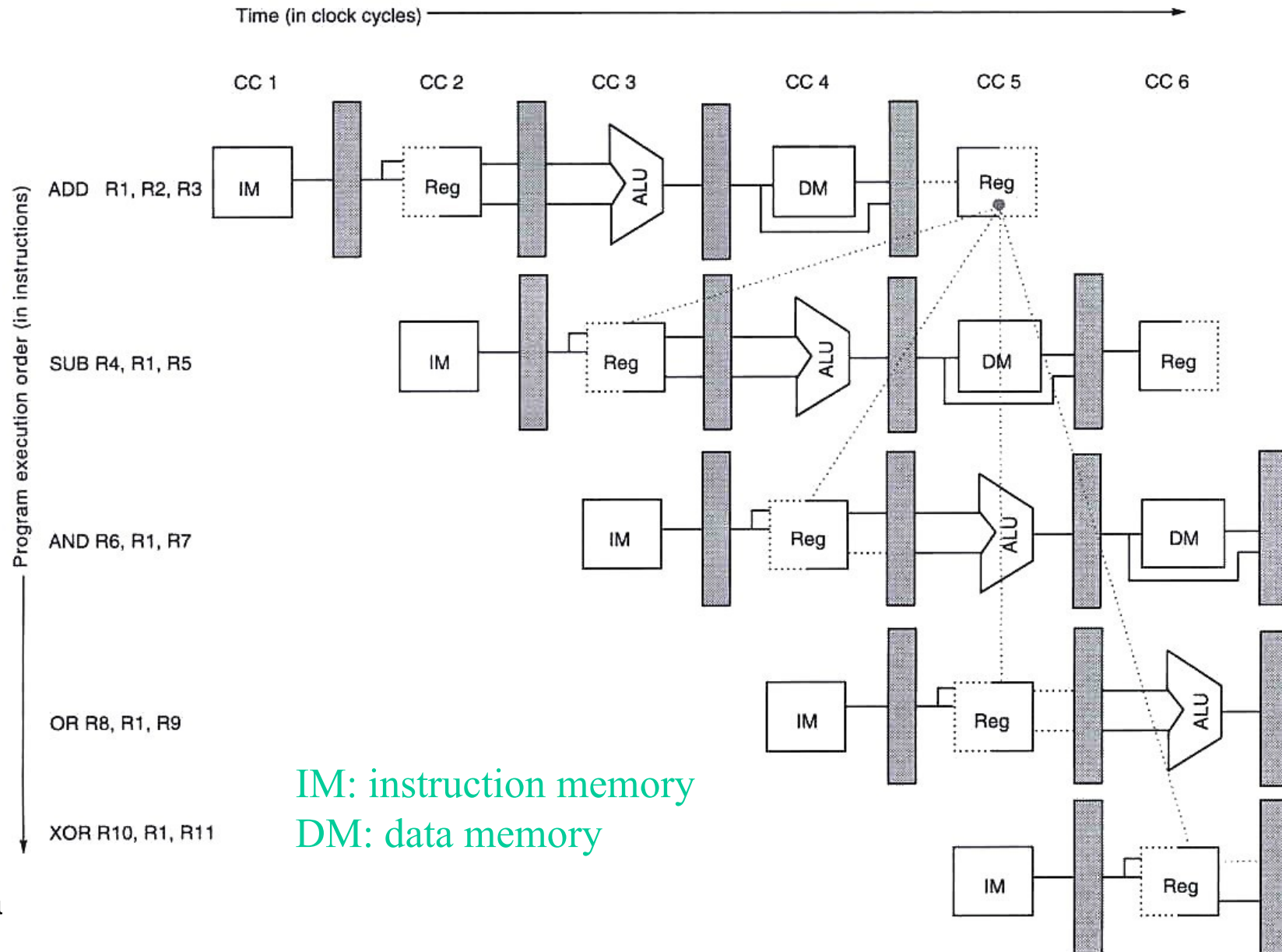
- Data hazards occur when the pipeline changes the order of read/write accesses to operands.

Example

I1	ADD R1, R2, R3
I2	SUB R4, R1, R5
I3	AND R6, R1, R7
I4	OR R8, R1, R9
I5	XOR R10, R1, R11

All instructions I2-I5 use the result of ADD operation (I1)

Pipelining the instruction stream (R1 is not ready until cc5)



Solution 1 to deal with data hazards:

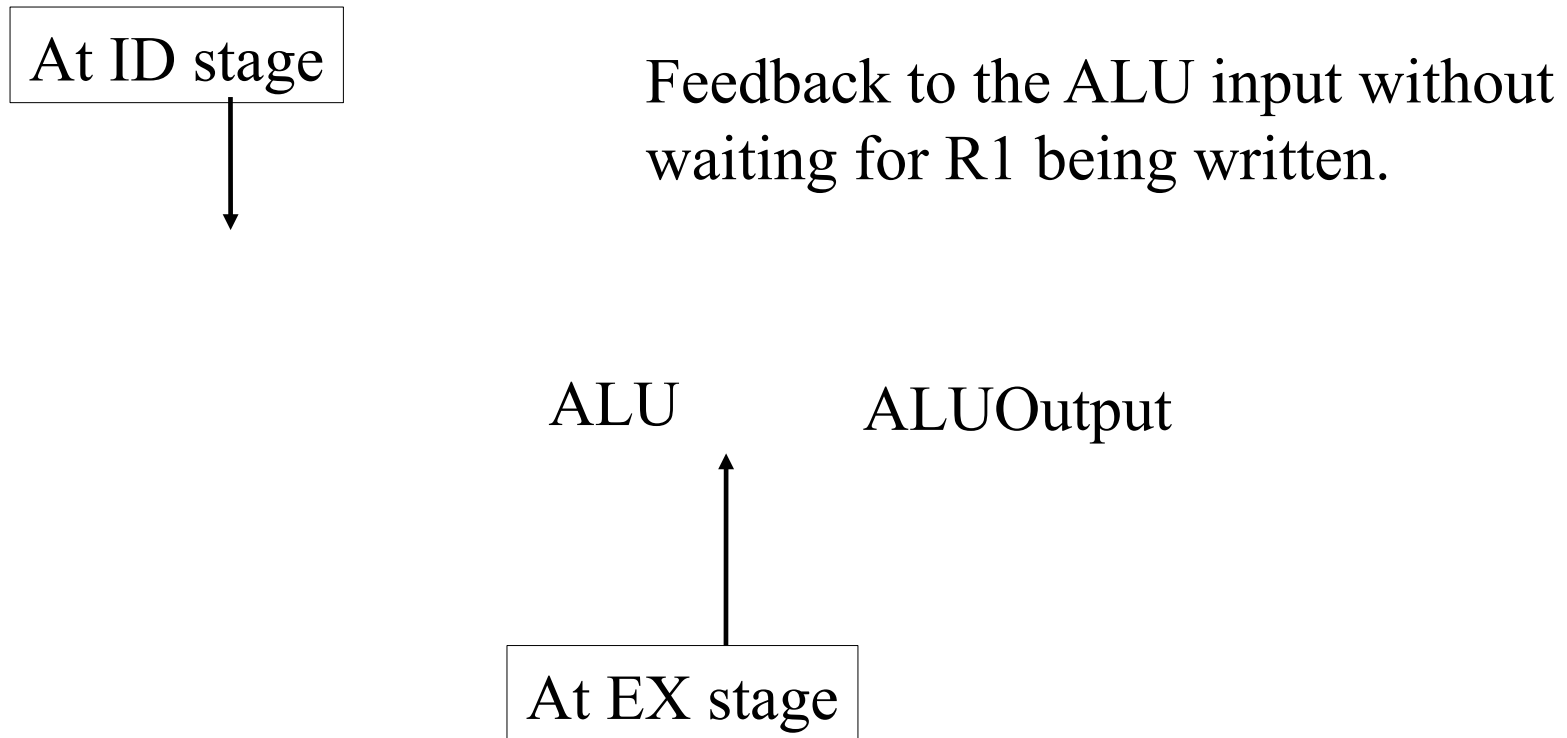
Forwarding – a hardware technique

- ALU result from EX/MEM registers is always fed back to the ALU input latches. (latches: pipeline registers)
- If forwarding hardware detects that the previous ALU operation has written the register corresponding to a source for the current ALU operation, control logic selects the forwarded result as the ALU input rather than the value read from the register file.

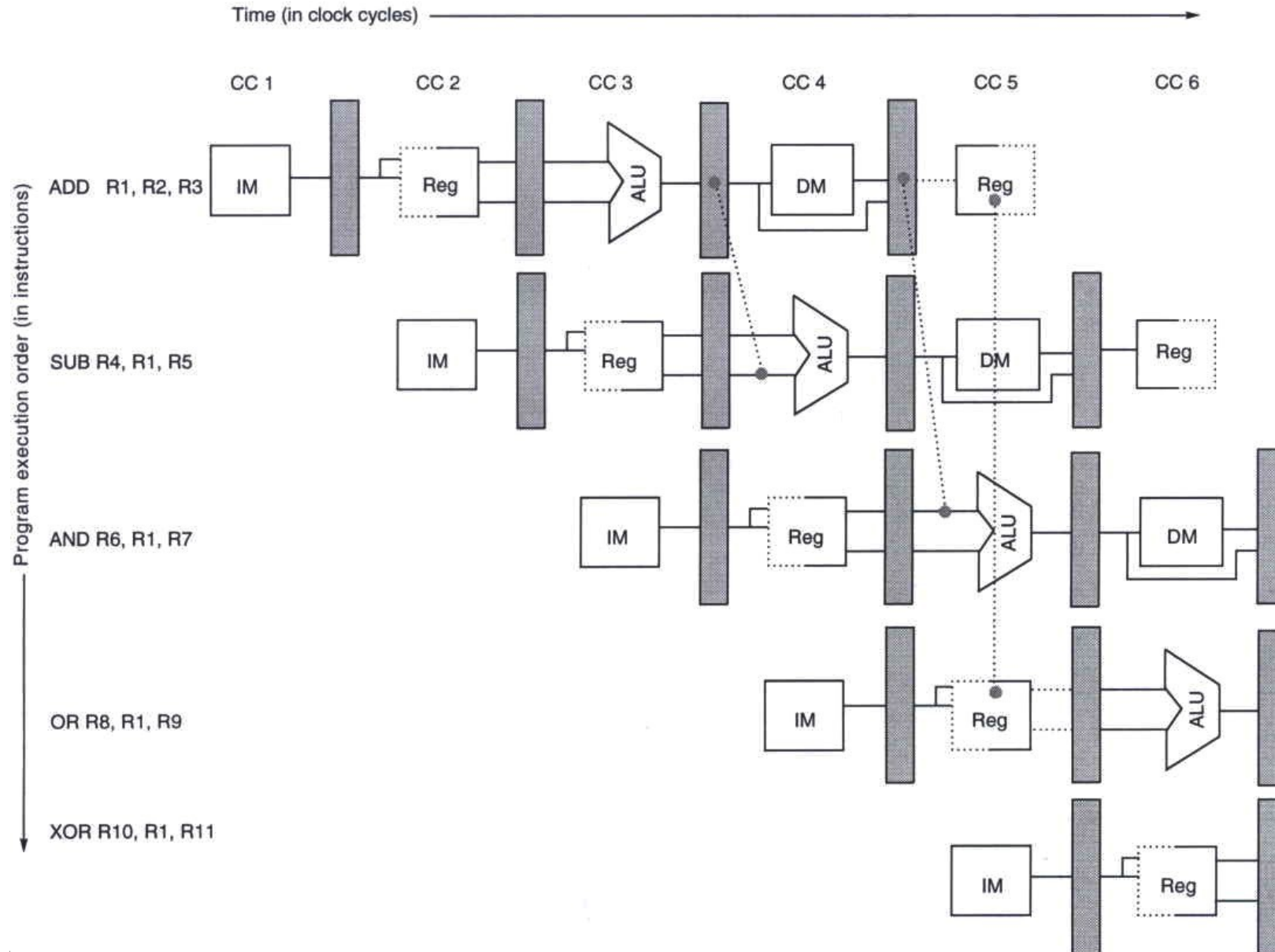
Solution 1 to deal with data hazards:

Forwarding – a hardware technique

- It looks like below. *Add R1, R2, R3*



Instructions use forwarding paths to avoid the data hazard.



Data Hazards (continued)

Example

Instruction sequence:

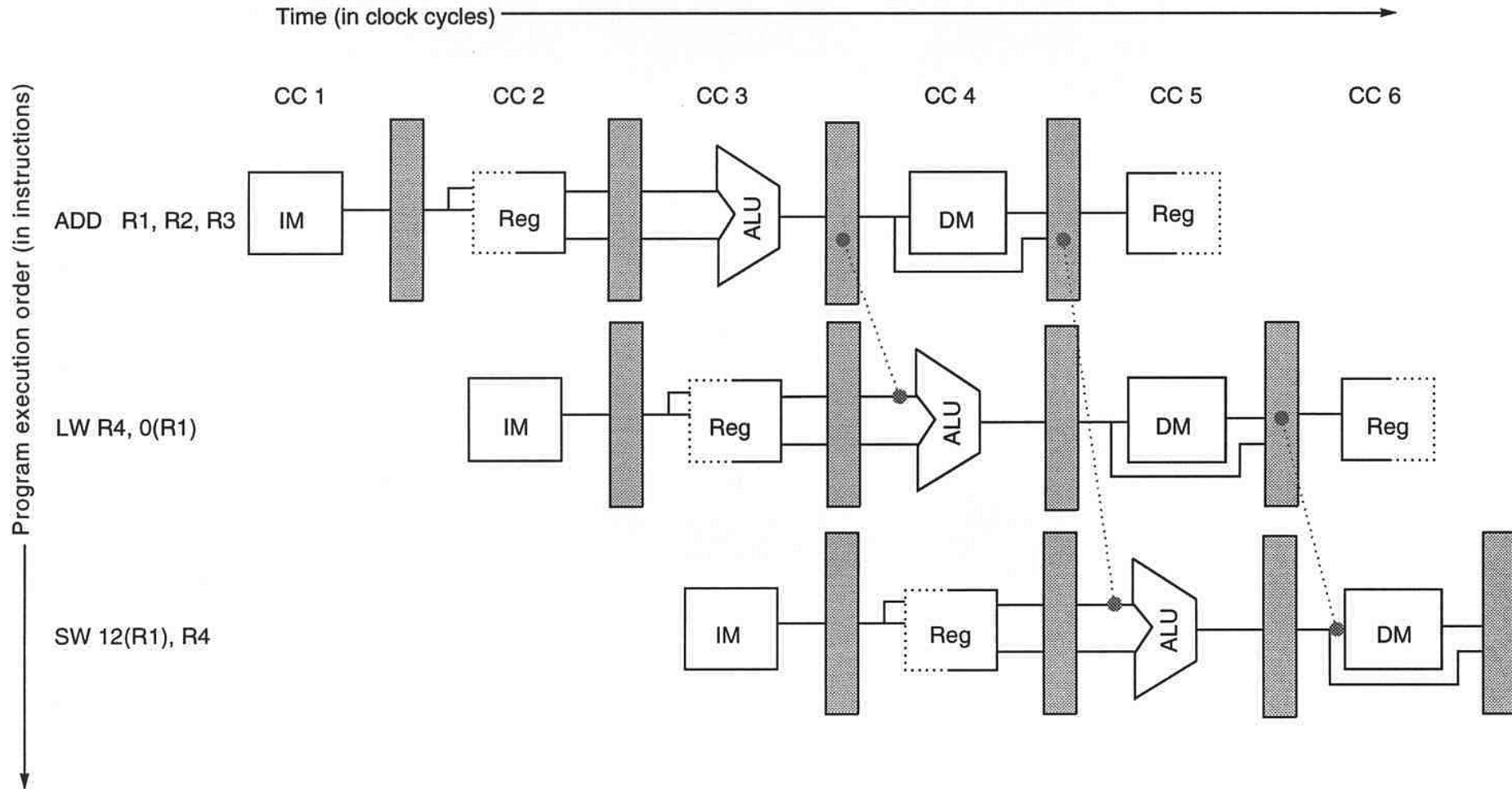
ADD R1, R2, R3

LW R4, 0(R1)

SW 12(R1), R4

To prevent a stall, need to forward values of R1 and R4 from the pipeline registers to the ALU and data memory inputs.

Forwarding the required operands to load and store.

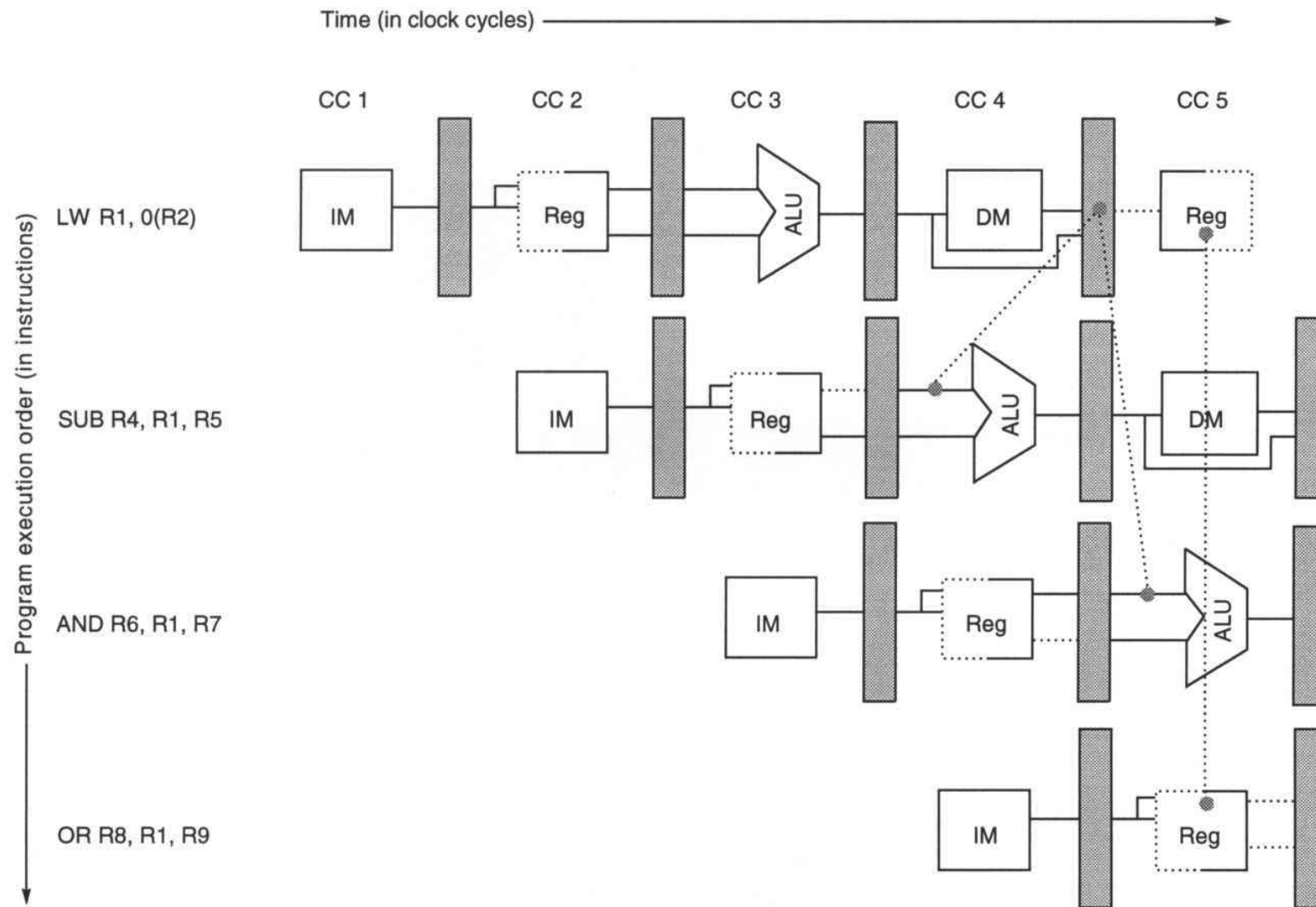


Data hazards require stalls

- Data hazards can not be fully handled by the forwarding technique in the following sequence of instructions

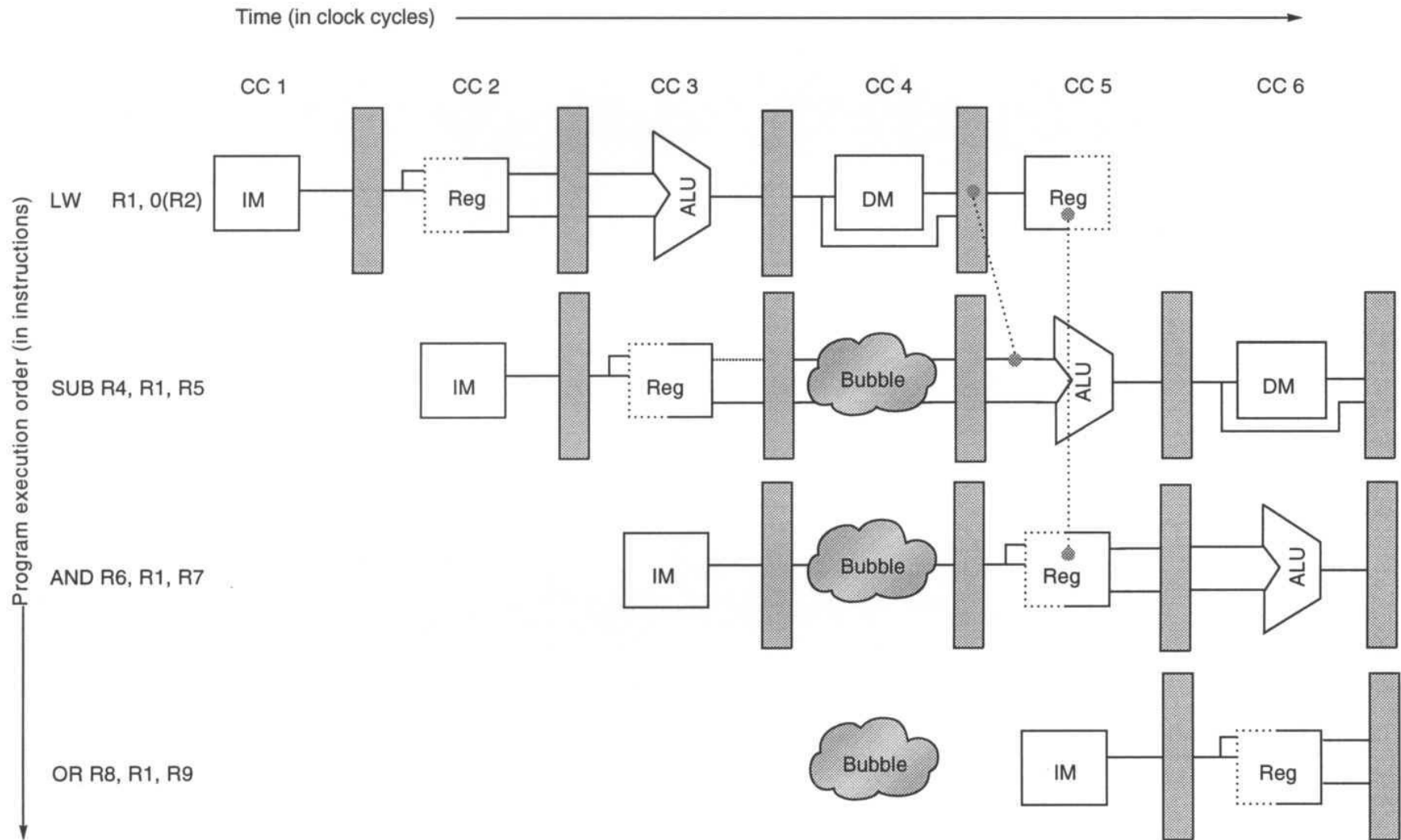
LW	R1, 0(R2)
SUB	R4, R1, R5
AND	R6, R1, R7
OR	R8, R1, R9

We have to stall the pipeline to avoid hazards.



The LW instruction does not produce data until the end of cc4(MEM), while SUB instruction needs the data at the beginning of cc4.

Pipeline interlock: to preserve correct execution pattern



Solution 2: Compiler scheduling to deal with data hazards

Code-generation pattern of $A=B+C$ produces stalls.

Instructions	clock cycles							
LW R1, B	IF	ID	EX	MEM	WB			
LW R2, C		IF	ID	EX	MEM	WB		
ADD R3, R1, R2			IF	ID	stall	EX	MEM	WB
SW A, R3				IF	stall	ID	EX	MEM WB

- Compiler rearrange a code sequence to eliminate data hazards

Example: Generate DLX code that avoids stalls for the sequence of $a=b+c$; and then $d=e-f$ in the 5-stage pipelined machine.

code: LW R2, b

LW R3, c

LW R5, e ** swap instructions

Add R1, R2, R3

LW R6, f

SW a, R1 ** load/store exchange

SUB R4, R5, R6

SW d, R4

** Out of sequence to avoid stalls

Data hazard classification

Two instructions i and j , with i occurring before j

- **RAW (*Read After Write*)**: j tries to read a source before i has written to it. Very common
- **WAW (*Write After Write*)**: j tries to write an operand before it is written by i . Writes end up in the wrong order;
- **WAR (*Write After Read*)**: j tries to write a destination before it is read by i .

Summary of the lecture

- Performance issues in pipelined processor
 - Equations for speedup from pipelining
 - Pipeline stall cycles degrade performance gained from pipelining
- Data hazards
 - Forwarding technique to reduce stall cycles
 - Compiler scheduling technique in dealing with data hazards

End of the lecture

Questions and exercises-1

- What are performance issues in pipelined machines?
- What are pipeline hazards?
- How to use various techniques to avoid data hazards in a pipelined machine?

End of the lecture

Questions and exercises-2

- Draw the pipeline timing table of the following instruction sequence running in a 5-stage DLX pipelined machine and identify possible data hazards

LW **R2**, 100(R1)

ADD R4, **R2**, R5

OR R6, **R2**, R7

XOR R8, **R2**, R9

- Adding the forwarding mechanism in the pipeline, calculate the performance gain from pipelining. Note: without pipelining, all instructions require 5 clock cycles in execution.